

Kubernetes Autoscaling Overview

Martina Baiardi
m.baiardi@unibo.it

31/05/2023

① Autoscaling

Che cosa significa Autoscaling?
Qual è l'utilità dell'Autoscaling?
I vantaggi dell'Autoscaling

② Autoscaling in Kubernetes

Metriche
Horizontal Pod Autoscaler
Vertical Pod Autoscaler
Cluster Autoscaler

③ Esempio pratico con Horizontal Pod Autoscaler

Installazione di un Cluster con Minikube
Dispiegamento di un HPA
Incremento del carico verso il Deployment

① Autoscaling

Che cosa significa Autoscaling?

Qual è l'utilità dell'Autoscaling?

I vantaggi dell'Autoscaling

② Autoscaling in Kubernetes

Metriche

Horizontal Pod Autoscaler

Vertical Pod Autoscaler

Cluster Autoscaler

③ Esempio pratico con Horizontal Pod Autoscaler

Installazione di un Cluster con Minikube

Dispiegamento di un HPA

Incremento del carico verso il Deployment

Che cosa significa Autoscaling?

Autoscaling

Tecnica utilizzata nel cloud computing per incrementare e/o ridurre in modo dinamico l'ammontare di risorse computazionali messe a disposizione da una server farm.

Qual è l'utilità dell'Autoscaling?

Quando si fa uso del cloud computing, si acquistano solamente le risorse necessarie per l'uso che si deve fare con i servizi che vengono messi a disposizione.

Il calcolo delle risorse necessarie non è semplice, perchè non prevedibile a priori. Infatti, il traffico può subire dei picchi al variare dell'orario e del giorno in questione.

Queste variazioni potrebbero non essere supportate dalle risorse che sono state acquistate dal cloud provider e di conseguenza i servizi esposti potrebbero subire una perdita di prestazioni e dare un disservizio agli utenti che vorrebbero utilizzarlo.

Qual è l'utilità dell'Autoscaling?

Per cercare di evitare queste situazioni spiacevoli si potrebbe pensare di acquistare dai cloud provider un numero sovra-elevato di risorse per poter garantire sempre disponibilità del servizio anche in momenti di traffico eccezionale.

Questo approccio però, che garantirebbe il funzionamento del servizio, ha dei risvolti negativi:

- le risorse che vengono acquistate sono utili solamente nel momento del picco, mentre in momenti di traffico ordinario sono sprecate
- il costo per chi offre il servizio è molto elevato e potrebbe non essere in grado di sostenerlo a lungo termine.

Qual è l'utilità dell'Autoscaling?

La soluzione? Autoscaling

Vengono acquistate dal cloud provider solamente le risorse necessarie a supportare un traffico ordinario verso il proprio servizio e ci si affida ad un servizio di Autoscaling che consente di rilevare il carico di lavoro attuale per aumentare/ridurre le risorse in base alle necessità in modo da pagare solamente quelle effettivamente utilizzate quando necessarie.

I vantaggi dell'Autoscaling

I vantaggi dell'autoscaling sono molteplici, tra i quali:

- 1 Le aziende che fanno uso di cloud privati possono spegnere e/o mandare in standby le macchine non necessarie per poter risparmiare sui costi dell'elettricità.
- 2 Le aziende che fanno uso di cloud pubblici possono ridurre i loro costi pagando solamente le risorse utilizzate quando necessarie.
- 3 Le soluzioni di Autoscaling (come quella offerta da Amazon Web Services) consente anche di sostituire le istanze definite *unhealthy*, ossia che hanno tempi di risposta medi troppo elevati, per evitare fallimenti dei servizi offerti.

① Autoscaling

Che cosa significa Autoscaling?

Qual è l'utilità dell'Autoscaling?

I vantaggi dell'Autoscaling

② Autoscaling in Kubernetes

Metriche

Horizontal Pod Autoscaler

Vertical Pod Autoscaler

Cluster Autoscaler

③ Esempio pratico con Horizontal Pod Autoscaler

Installazione di un Cluster con Minikube

Dispiegamento di un HPA

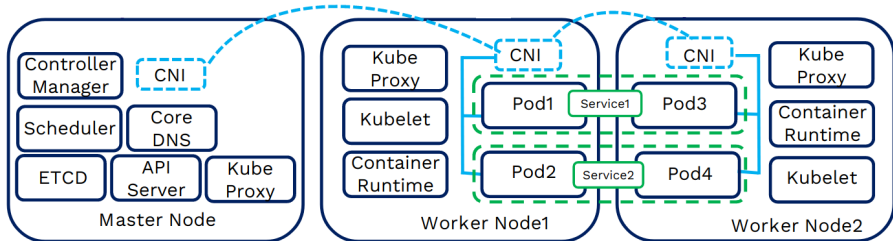
Incremento del carico verso il Deployment

Autoscaling in Kubernetes

Kubernetes offre tre strumenti per effettuare autoscaling

- ① **Horizontal Pod Autoscaler**
- ② **Vertical Pod Autoscaler**
- ③ **Cluster Autoscaler**

Kubernetes Overview

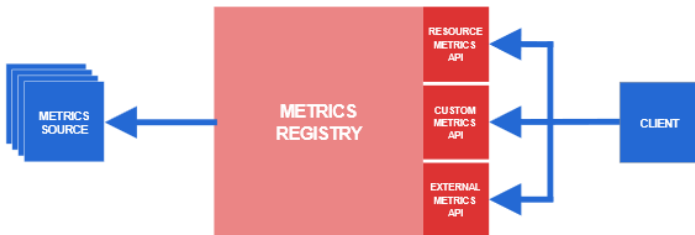


In Kubernetes le risorse computazionali come *memoria* e *cpu* sono chiamate **metriche**. Per poter monitorare le metriche dei vari componenti del cluster come Nodi e Pods, il Control Plane di Kubernetes, all'interno del Master Node, espone varie API poter gestire e monitorare il cluster.

Il componente del Control Plane che espone le API riguardanti le metriche si chiama **Metrics Registry**.

Kubernetes distingue tre tipologie di metriche all'interno del cluster:

- 1 **Metriche di Risorsa:** le quali comprendono memoria e cpu riguardanti `Nodi` e `Pods`. Queste metriche vengono esposte sull'API `metrics.k8s.io`.
- 2 **Metriche Custom:** osservano le metriche di altre risorse Kubernetes che non sono comprese dalle metriche di tipo risorsa. Queste informazioni sono esposte sull'API `custom.metrics.k8s.io`.
- 3 **Metriche Esterne:** osservano metriche non associate ad oggetti Kubernetes, come ad esempio il numero di richieste effettuate. Queste informazioni sono esposte sull'API `external.metrics.k8s.io`.



Per poter osservare le metriche è necessario inserire all'interno del cluster il `Metrics Registry` che non è compreso nell'installazione di default: noi useremo il **Metrics Server**.

Il `Metrics Server` è il componente che controlla ed espone tutte le informazioni riguardanti le metriche sulle API opportune. Per esporre tali dati, il server si appoggia su un altro componente chiamato **Metrics Collector**, il quale si occupa di raccogliere le informazioni richieste.

- 1 Le metriche di tipo risorsa sono osservate tramite il collector **cAdvisor (Container Advisor)** il quale viene eseguito di default all'interno di ciascun Worker Node del cluster.
- 2 Le metriche di tipo custom e quelle esterne invece sono raccolte sfruttando collector esterni come ad esempio **Prometheus, Datadog e Google Stackdriver**. Questi controller non sono presenti di default all'interno del cluster e richiedono un'opportuna configurazione.

Ogni collector esterno propone anche il proprio API Server per poter interagire con le informazioni raccolte.

CPU

Le risorse di tipo CPU sono espresse in *millesimi di cpu*. Una CPU per Kubernetes è equivalente ad un core per i Cloud Provider (o equivalentemente ad un Hyperthread per i processori fisici).

Una metrica cpu può essere espressa come valore percentuale oppure come millesimi:

$$0.1 = 10\% = 100m$$

La quantità espressa è da considerarsi assoluta e non relativa, questo significa che $100m$ rappresenta la stessa quantità sia su una macchina single-core sia su una da 48 core.

Memoria

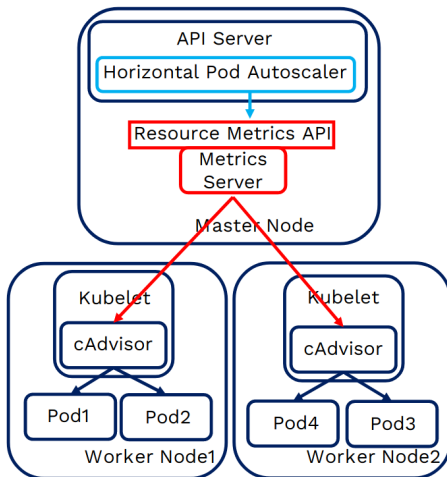
L'unità di misura per indicare la memoria è il byte. La quantità può essere espressa come valore intero oppure specificando un suffisso che ne specifica l'ordine di grandezza:

$$128974848 = 129e6 = 129M = 123Mi$$

Horizontal Pod Autoscaler

L'**Horizontal Pod Autoscaler (HPA)** è il componente che consente di scalare automaticamente il numero di repliche di un **Pod** sia in incremento che in riduzione sulla base dell'osservazione di determinate metriche.

Horizontal Pod Autoscaler



Horizontal Pod Autoscaler

L'algoritmo utilizzato da questo componente è molto semplice e basa il suo funzionamento sull'osservazione dei valori di determinate metriche, specificate nel momento della creazione di una risorsa HPA.

$$desiredReplicas = \text{ceil} \left[currentReplicas * \frac{currentMetricValue}{desiredMetricValue} \right]$$

La verifica delle metriche viene fatta con un loop di controllo effettuato ogni 15 secondi, ma la configurazione di default può essere modificata in base alle necessità d'uso del cluster.

Horizontal Pod Autoscaler

È possibile creare una risorsa HPA tramite la definizione di un file Yaml oppure attraverso l'utilizzo del comando `kubectl autoscale`.

```
kubectl autoscale deployment foo --max=5 --cpu-percent=80
```

Il comando cerca il `Deployment` **foo** all'interno del cluster e gli associa una risorsa HPA che cerca di mantenere l'occupazione della cpu inferiore all'80% della capacità massima dei pod, aumentando e diminuendo automaticamente il numero di `Pod` del `Deployment` fino ad un massimo di 5, se necessario.

Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

scaleTargetRef indica qual è il Deployment che dovrà essere monitorato dall'istanza dell'Horizontal Pod Autoscaler

Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

minReplicas e maxReplicas
indicano qual è il numero minimo
e massimo di Pod che verranno
replicati dall'Horizontal Pod
Autoscaler

Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: php-apache
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

metrics indica qual è la risorsa che deve essere monitorata e specifica anche la soglia che deve essere rispettata

Il **Vertical Pod Autoscaler (VPA)** è il componente che consente di incrementare le risorse richieste da un Pod al Nodo su cui è dispiegato in termini di cpu e memoria.

Allo stato attuale, il VPA può essere utilizzato solamente tramite il **Metrics Server**.

Il VPA si compone di tre elementi:

- 1 **Recommender:** componente che osserva l'utilizzo attuale delle risorse da parte del `Pod` e verifica quali sono i valori massimi di risorse messi a disposizione dal `Nodo` sottostante.
- 2 **Updater:** siccome un `Pod` non può essere aggiornato mentre è in esecuzione, questo componente si occupa di eliminare l'istanza precedente e dispiegarne una nuova, che comprenderà la sua richiesta di risorse aggiornata.
- 3 **Admission Controller:** componente che si occupa di incrementare le risorse richieste dal nuovo `Pod` che verrà dispiegato dal VPA.

Allo stato attuale, è ancora in fase di sperimentazione la possibilità di aggiornare le risorse utilizzate da un `Pod` a run-time, evitando così l'onere di dover eliminare l'istanza precedente e crearne una nuova aggiornata.

Per questo motivo il VPA è ancora in fase di `beta`.

Componente che consente di aumentare e/o diminuire il numero di nodi di un cluster. Il Cluster Autoscaler effettua un loop tra due task principali:

- 1 **Task 1:** Viene controllata l'eventuale presenza di `Pods` che non possono essere schedulati (eventualmente a causa di mancanza di risorse disponibili sui nodi) e viene valutato se l'introduzione di un nuovo `Nodo` ne consentirebbe lo sbloccamento. In caso di risposta affermativa il nodo viene aggiunto al cluster.
- 2 **Task 2:** Viene valutato se è possibile dispiegare tutti i `Pods` attualmente attivi in un numero inferiore di `Nodi`, in caso di risposta affermativa il numero di `Nodi` viene ridotto e i `Pods` rischedulati.

Ad oggi il `Cluster Autoscaler` è un componente che può essere utilizzato solamente all'interno degli ambienti cloud

① Autoscaling

Che cosa significa Autoscaling?

Qual è l'utilità dell'Autoscaling?

I vantaggi dell'Autoscaling

② Autoscaling in Kubernetes

Metriche

Horizontal Pod Autoscaler

Vertical Pod Autoscaler

Cluster Autoscaler

③ Esempio pratico con Horizontal Pod Autoscaler

Installazione di un Cluster con Minikube

Dispiegamento di un HPA

Incremento del carico verso il Deployment

Minikube

`Minikube` è un'installazione locale di `Kubernetes` utile per fare testing dei suoi componenti. L'installazione è molto semplice e configura in automatico tutti gli strumenti necessari per poter utilizzare il cluster. Il cluster installato da `Minikube` è composto da un solo nodo e lo strumento è predisposto per l'installazione automatica di plugin attraverso l'uso dei `minikube addons`.

Installazione di Minikube

Installazione di Minikube

È possibile effettuare l'installazione di `Minikube` sulla propria macchina seguendo le istruzioni riportate in [questo](#) link.

Creazione del Cluster

Per creare il cluster è possibile eseguire il comando:

```
minikube start --extra-config=kubelet.housekeeping-interval=10s
```

Attenzione: è importante passare il flag `housekeeping-interval` al comando perchè serve per far funzionare il Metrics Server correttamente all'interno del cluster.

Installazione del comando `kubectl`

`kubectl` è il comando che consente di interagire con un cluster Kubernetes da remoto, non solo consente di monitorarne il funzionamento ma permette di mettere in esecuzione nuovi servizi da qualsiasi macchina remota. Può essere installato seguendo la [guida ufficiale](#).

Per poter funzionare, `kubectl` si appoggia ad una configurazione presente nel percorso `$HOME/.kube/config` che contiene le informazioni necessarie per poter interagire con uno o più cluster in cui l'utente ha accesso.

Nota

Minikube, in automatico, configura `kubectl` per poter comunicare con l'istanza di Kubernetes appena creata.

Installazione del Metrics Server

Minikube Addons

Minikube fornisce un insieme di servizi aggiuntivi pre-configurati che è possibile installare all'interno del cluster, è possibile visualizzare quelli disponibili con il comando:

```
minikube addons list
```

Metrics Server

Per installare il Metrics Server all'interno del cluster basta eseguire il comando:

```
minikube addons enable metrics-server
```

Dispiegamento di un Deployment

Dispieghiamo ora il seguente Deployment fornito da google per mostrare il funzionamento dell'HPA:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
spec:
  selector:
    matchLabels:
      run: php-apache
  replicas: 1
  template:
    metadata:
      labels:
        run: php-apache
    spec:
      containers:
        - name: php-apache
          image: registry.k8s.io/hpa-example
          ports:
            - containerPort: 80
      resources:
        limits:
          cpu: 500m
        requests:
          cpu: 200m
```

Il seguente Deployment genera un Pod contenente una immagine php-apache che effettua un calcolo computazionale molto oneroso ad ogni richiesta ricevuta in ingresso.

Dispiegamento di un Deployment

Dispieghiamo ora il seguente Deployment fornito da google per mostrare il funzionamento dell'HPA:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
spec:
  selector:
    matchLabels:
      run: php-apache
  replicas: 1
  template:
    metadata:
      labels:
        run: php-apache
    spec:
      containers:
        - name: php-apache
          image: registry.k8s.io/hpa-example
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: 500m
            requests:
              cpu: 200m
```

Inoltre, vengono definite le risorse minime e massime a cui il Pod può accedere sul Nodo in cui è eseguito:

- richiede almeno 200 millesimi di cpu disponibili
- userà al più 500 millesimi di cpu

Dispiegamento di un Deployment

Possiamo dispiegare questo Deployment all'interno del nostro cluster con il comando:

```
kubectl apply -f https://k8s.io/examples/application/php-apache.yaml
```

Dispiegamento dell'Horizontal Pod Autoscaler

Una volta che i Pods sono in stato `Running`, possiamo procedere con la creazione della risorsa `Horizontal Pod Autoscaler`, che possiamo fare direttamente con il comando:

```
kubectl autoscale deployment php-apache --cpu-  
percent=50 --min=1 --max=10
```

Verrà creata all'interno del cluster una risorsa HPA che si mette in ascolto della cpu utilizzata dal Deployment `php-apache`, in particolare osserverà quando l'uso supererà la soglia del 50% e procederà con lo scaling delle istanze.

Le repliche dei Pod potranno variare da un minimo di una ad un massimo di 10.

Incremento del carico verso il Deployment

In un nuovo terminale possiamo eseguire questo comando:

```
kubectl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- http://php-apache; done"
```

Questo consente di creare un Pod che effettua un Polling di chiamate verso il Deployment che abbiamo esposto nel Cluster.

Incremento del carico verso il Deployment

Eseguendo il comando:

```
kubectl get hpa php-apache --watch
```

Possiamo verificare che il nostro Horizontal Pod Autoscaler, dopo un po' di minuti, inizierà ad incrementare le istanze dei `Pod` per consentire di rispondere a tutte le richieste che si stanno verificando.

Incremento del carico verso il Deployment

Terminando il `Pod` creato per effettuare le richieste al nostro `Deployment`, possiamo verificare che dopo un po' di minuti il numero di repliche tornerà a scendere in automatico, fino a tornare ad una sola replica come nella situazione iniziale.

Inizialmente nessuna richiesta viene effettuata al Pod ...

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	0%/50%	1	10	1	13d

Inizia ad aumentare il carico di lavoro del Pod ...

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	0%/50%	1	10	1	13d
php-apache	Deployment/php-apache	211%/50%	1	10	1	13d

L'Horizontal Pod Autoscaler aumenta il numero di Pods ...

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	0%/50%	1	10	1	13d
php-apache	Deployment/php-apache	211%/50%	1	10	1	13d
php-apache	Deployment/php-apache	224%/50%	1	10	4	13d
php-apache	Deployment/php-apache	81%/50%	1	10	5	13d

Il carico di lavoro viene effettivamente ridotto dopo aver allocato nuove istanze ...

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	0%/50%	1	10	1	13d
php-apache	Deployment/php-apache	211%/50%	1	10	1	13d
php-apache	Deployment/php-apache	224%/50%	1	10	4	13d
php-apache	Deployment/php-apache	81%/50%	1	10	5	13d

Le richieste terminano e il numero delle repliche si adatta di conseguenza ...

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	11%/50%	1	10	5	13d
php-apache	Deployment/php-apache	0%/50%	1	10	5	13d
php-apache	Deployment/php-apache	0%/50%	1	10	2	13d
php-apache	Deployment/php-apache	0%/50%	1	10	1	13d